CS {4/6}290 & ECE {4/6}100 - Spring 2021

Homework 2: Caches

Dr. Thomas Conte

Due: February 12th, 2021 @ 11:59 PM

Rules

- **This is an individual assignment. ABSOLUTELY NO COLLABORATION IS PERMITTED**. All cases of honor code violations will be reported to the Dean of Students.
- The due date at the top of the assignment is final. Late assignments will not be accepted.
- Please use office hours for getting your questions answered. If you are unable to make it to office hours, please email the TAs.
- Make sure you **show your work and give explanations** for all questions unless otherwise stated.
- All answers must be **handwritten or typed**, and an electronic copy (PDF format) uploaded to Canvas before the due date.
- **Make sure your answers are written within the boxes we have provided. We will be using Gradescope for grading and answers outside the boxes will NOT be recognized by the system.**
- **There will be a point penalty for not following the above rule.**

Question 0 – Cache Warmup

(a) Assume a cache with (C = 10, B = 6, S = 0). Given that this cache implements a write-allocate, write-back policy, and system addresses are 32-bits wide:

(i) How many bits make up the *Tag*, *Index*, and *Block Offset* of an address accessing this Cache? (i.e. show the address split for accessing this cache)

Block offset = B = 6 bits
Index = (C – B) – S = 4 bits
Tag = address length – (block offset bits + index bits) = 22

(ii) How many bits are there in the *Data Store* of the Cache?

2^C bytes of data store in the cache, 2^3 bits in a byte
$2^C * 2^3 = 2^{13}$ bits of data store

(iii) How many bits are there in the *Tag Store* of this Cache?

22 bits in the tag per block
2^C bytes in cache / 2^B bytes per block = 2^{C-B} blocks in the cache
2^{C-B} blocks in cache * (22 bits of tag per block + 1 valid bit + 1 dirty bit) = 384 bits of tag store

(b) Consider the C, B and S parameters of a Cache. What happens to compulsory, capacity and conflict misses if only the following parameter is changed (the other two are kept the same)?

(i) C is increased

Compulsory – no change, Capacity – decreases, Conflict – decreases

(ii) S is increased

Compulsory – no change, Capacity – no change, Conflict – decreases

(iii) B is increased

Compulsory – decreases, Capacity – no change, Conflict – decreases at first then increases when cache pollution occurs

Question 1 – Hand to Hand Caches

Consider the memory address trace below for a 32-bit address size system:

```
ST 0x0ECE4100
LD 0x0ECE6100
LD 0x00C1CD88
LD 0x0054B0FA
LD 0x00ABCD89
ST 0x00CDEF9A
LD 0x000CB017
LD 0x1002ED12
LD 0x1054B09B
LD 0x00ABCD88
LD 0x00EF7300
ST 0x000CB680
```

Hand simulate the above cache accesses for the *(C,B,S)* cache configurations (a)-(c). Assume that caches use the LRU replacement policy (where applicable) and implement the write-back, write-allocate scheme. **Answer questions (i)-(iii) for each of the configurations:**

(i) Indicate which accesses would cause a miss. If an access is a miss, identify the type of miss.
(ii) How many write-backs happen for this stream? **Explain why?**
(iii) If the cache policy was *write-through instead of write-back*, how many trips to main memory will there be for this stream? **Explain why?**

(a) (9, 4, 0) – **block offset = 4 bits, index = 5 bits, tag = 23**
(b) (9, 4, 4) – **block offset = 4 bits, index = 1 bits, tag = 27**
(c) (9, 4, 2) – **block offset = 4 bits, index = 3 bits, tag = 25**

Fill in this table to answer (i) {Hit : H, Miss : M} if (M) { Type}

Stream	(a)	(b)	(c)
ST 0x0ECE4100	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x0ECE6100	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x00C1CD88	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x0054B0FA	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x00ABCD89	M - Compulsory	M - Compulsory	M - Compulsory
ST 0x00CDEF9A	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x000CB017	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x1002ED12	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x1054B09B	M - Compulsory	M - Compulsory	M - Compulsory
LD 0x00ABCD88	H	H	H
LD 0x00EF7300	M - Compulsory	M - Compulsory	M - Compulsory
ST 0x000CB680	M - Compulsory	M - Compulsory	M - Compulsory

Space for Answer to Question 1 (ii) and (iii)

Stream (a)	Tag (23 bits)	Index (5 bits)	H/M & Type	WB
ST 0x0ECE4100	0x0ECE4 000	10000	M - Compulsory	Mark dirty
LD 0x0ECE6100	0x0ECE6 000	10000	M - Compulsory	Write back
LD 0x00C1CD88	0x00C1C 110	11000	M - Compulsory	
LD 0x0054B0FA	0x0054B 000	01111	M - Compulsory	
LD 0x00ABCD89	0x00ABC 110	11000	M - Compulsory	
ST 0x00CDEF9A	0x00CDE 111	11001	M - Compulsory	Mark dirty
LD 0x000CB017	0x000CB 000	00001	M - Compulsory	
LD 0x1002ED12	0x1002E 110	10001	M - Compulsory	
LD 0x1054B09B	0x1054B 000	01001	M - Compulsory	
LD 0x00ABCD88	0x00ABC 110	11000	H	
LD 0x00EF7300	0x00EF7 001	10000	M - Compulsory	
ST 0x000CB680	0x000CB 011	01000	M - Compulsory	Mark dirty

Number of write-backs = 1 (index 10000 has a write and then the dirty block is replaced)

Number of trips to main memory if write-through = 3 (1 write to memory for every store) + 11 (1 read for every miss, including write since write-allocate) = 14
Note: going from cache to memory, or memory to cache, counts as separate trips

Stream (b)	Tag (27 bits)	Index (1 bits)	H/M & Type	WB
ST 0x0ECE4100	0x0ECE41 000	0	M - Compulsory	Mark dirty
LD 0x0ECE6100	0x0ECE61 000	0	M - Compulsory	
LD 0x00C1CD88	0x00C1CD 100	0	M - Compulsory	
LD 0x0054B0FA	0x0054B0 111	1	M - Compulsory	
LD 0x00ABCD89	0x00ABCD 100	0	M - Compulsory	
ST 0x00CDEF9A	0x00CDEF 100	1	M - Compulsory	Mark dirty
LD 0x000CB017	0x000CB0 000	1	M - Compulsory	
LD 0x1002ED12	0x1002ED 000	1	M - Compulsory	
LD 0x1054B09B	0x1054B0 100	1	M - Compulsory	
LD 0x00ABCD88	0x00ABCD 100	0	H	
LD 0x00EF7300	0x00EF73 000	0	M - Compulsory	
ST 0x000CB680	0x000CB6 100	0	M - Compulsory	Mark dirty

Number of write-backs = 0 (no dirty blocks are replaced)

Number of trips to main memory if write-through = 3 (1 write to memory for every store) + 11 (1 read for every miss, including write since write-allocate) = 14
Note: going from cache to memory, or memory to cache, counts as separate trips

Stream (c)	Tag (25 bits)	Index (3 bits)	H/M & Type	WB
ST 0x0ECE4100	0x0ECE41 0	000	M – Compulsory	Mark dirty
LD 0x0ECE6100	0x0ECE61 0	000	M – Compulsory	
LD 0x00C1CD88	0x00C1CD 1	000	M – Compulsory	
LD 0x0054B0FA	0x0054B0 1	111	M – Compulsory	
LD 0x00ABCD89	0x00ABCD 1	000	M – Compulsory	
ST 0x00CDEF9A	0x00CDEF 1	001	M – Compulsory	Mark dirty
LD 0x000CB017	0x000CB0 0	001	M – Compulsory	
LD 0x1002ED12	0x1002ED 0	001	M – Compulsory	
LD 0x1054B09B	0x1054B0 1	001	M – Compulsory	
LD 0x00ABCD88	0x00ABCD 1	000	H	
LD 0x00EF7300	0x00EF73 0	000	M – Compulsory	Write back
ST 0x000CB680	0x000CB6 1	000	M - Compulsory	Mark dirty

Number of write-backs = 1 (index 000 has a full set, and using LRU means the dirty block is the one replaced)

Number of trips to main memory if write-through = 3 (1 write to memory for every store) + 11 (1 read for every miss, including write since write-allocate) = 14

Note: going from cache to memory, or memory to cache, counts as separate trips

Question 2 – Cache Design

Given the memory trace from problem 1 with parameter B = 4:

(a) What is the smallest direct-mapped cache for which no misses besides *compulsory* occur?
(b) What is the smallest Fully-Associative cache for which no misses besides *compulsory* occur?
(c) What is the smallest possible Set-Associative Cache (Not fully associative) that completely avoids *Capacity* and *Conflict* misses? (Find the minimum C and S values to satisfy the constraint)

Please show your work and explain

Space for Answer to Question 2

a) Due to address stream, there is already mainly compulsory misses, and our only repeat tag likely is the 5th and 10th access. If you look at the breakdown of stream (b) in Question 1, you'll notice when 1 index bit is used, 5th and 10th access have index '1' and the accesses between them have '0', so we can essentially use two blocks to avoid what would be the only non-compulsory miss.

Direct-mapped means S = 0, and given B = 4
Therefore, you can use (5, 4, 0) as the cache design, C = 5 defines your minimum cache size. 2^5 bytes

b) for fully-associative, cache just needs to be big enough to hold the 5th access until the 10th has occurred. Therefore, it needs to be at least 5 blocks large.

2^C bytes in cache / 2^B bytes per block >= 5 blocks in cache

So C = 7

c) We know from a) that (5, 4, 0) works, a cache with only 2 blocks. If we need S > 0, we try S = 1. If we use (5, 4, 1) we would only have 1 set, and we would replace our 5th access before the 10th occurs. Therefore, we need at least 2 sets/indexes in order to avoid non-compulsory misses like in part a). So a cache defined by (6, 4, 1) works. C = 6

Question 3 – Cache Performance

A given memory hierarchy initially consists of an L1 cache and main memory. The L1 cache has a hit time of 2 cycles, and for a given workload a miss ratio (rate) of 15 percent. It takes 100 cycles to access main memory on average.

(a) What is the average access time for this system? **Show your work**

$AAT = HT_1 + MR_1 * MP$
$AAT = 2 + 0.15*100 = 17$ cycles

You are the chief memory architect for ACME computers and decide to improve your systems performance by considering the options listed in (b)-(d)

(b) Modification Option 1: Add a victim cache to the hierarchy. The victim cache has a miss ratio of 0.7. What is the average access time of the system now?

$AAT = HT_1 + MR_1 * MR_{vc} * MP$
$AAT = 2 + 0.15 * 0.7 * 100 = 12.5$ cycles

(c) Modification Option 2: Add a second level cache (L2) with the following characteristics:
Hit Time = 8 Cycles, Miss Rate = 5 percent

$AAT = HT_1 + MR_1 * (HT_2 + MR_2 * MP)$
$AAT = 2 + 0.15 * (8 + 0.05 * 100) = 3.95$ cycles

(d) Modification Option 3: You give up improving your system cache entirely and instead want to tell your main memory supplier to make faster memory… Yikes! How fast must main memory be such that when used with the original system the overall AAT is less than 4 cycles? *(It is okay to have a fractional value for memory latency for this problem)*

$AAT = HT_1 + MR_1 * MP$
$HT_1 + MR_1 * MP <= 4$
$2 + 0.15 * MP <= 4$
$MP <= 13.333$ cycles

(e) Which option do you choose after evaluating all of them or do you stick with your original memory system? **Please explain**

Adding a second cache L2 is probably the best option, since it is faster than VC and original design. It will be easier to "just" add an L2 than waiting on main memory to be speed up. Making main memory faster is increasingly difficult due to complications when shrinking capacitors beyond a certain limit.